

LAPORAN TUGAS BESAR TEORI BAHASA FORMAL DAN AUTOMATA

IF3170 Teori Bahasa Formal dan Automata

Milestone 3 : Semantic Analysis



Disusun Oleh:

Indah Novita Tangdililing (13523047)

Muhammad Flthra Rizki (13523049)

Sakti Bimasena (13523053)

Muhammad Timur Kanigara (13523055)

Kefas Kurnia Jonathan (13523113)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

BANDUNG

2025

DAFTAR ISI

DAFTAR ISI.....	2
1. Landasan Teori.....	3
1.1. Definisi Semantic Analysis.....	3
1.2. Attributed Grammar dan Abstract Syntax Tree (AST).....	3
1.3. Symbol Table.....	4
1.4. Visitor Pattern dan Type Checking.....	5
2. Perancangan dan Implementasi.....	5
2.1. Struktur Data Abstract Syntax Tree (AST).....	5
2.2. Implementasi Symbol Table.....	7
2.3. Pembangunan AST.....	8
2.4. Type Checker.....	11
3. Pengujian.....	17
3.1. Tes Scoping.....	17
3.2. Tes Array.....	18
3.3. Tes Error Duplikasi.....	21
3.4. Tes Error Scoping.....	21
3.5. Tes Error Undeclared.....	22
3.6. Tes Error Type.....	22
4. Kesimpulan dan Saran.....	24
4.1. Kesimpulan.....	24
4.2. Saran.....	24
5. Lampiran.....	26
6. Referensi.....	27

1. Landasan Teori

Beberapa teori dan konsep dasar yang digunakan dalam pengerjaan Milestone 3 meliputi analisis semantik, penggunaan *Attributed Grammar*, struktur *Symbol Table* serta mekanisme penelusuran tree menggunakan *Visitor Pattern*. Penjelasan lebih lengkapnya dapat dilihat pada sub-bab berikut.

1.1. Definisi *Semantic Analysis*

Analisis semantik atau *Semantic Analysis* merupakan fase ketiga dalam proses kompilasi, setelah *lexical analysis* dan *syntax analysis*. Parser (*syntax analysis*) memastikan kode program mematuhi aturan produksi grammar, akan tetapi program masih memungkinkan mengandung kesalahan makna atau logika yang tidak konsisten meskipun sintaksnya benar.

Pada tahap *Semantic Analysis*, tujuan utamanya adalah memeriksa konsistensi program, meliputi pemeriksaan tipe data (*type checking*), deklarasi variabel dan fungsi, lingkup (*scope*) variabel, serta aliran kontrol (*control flow*). Hasil dari proses ini adalah *Abstract Syntax Tree* (AST) yang telah disesuaikan dengan informasi semantik seperti tipe data dan referensi ke tabel simbol.

1.2. *Attributed Grammar* dan *Abstract Syntax Tree* (AST)

Attributed Grammar adalah sebuah konsep *grammar* yang dilengkapi dengan atribut dan aturan semantik untuk melakukan analisis makna. Aturan ini memungkinkan *compiler* untuk menghitung nilai atribut pada setiap *node* dalam *syntax tree*.

Dalam implementasinya, skema penerjemahannya menggunakan *Syntax-Directed Translation Scheme*) untuk membentuk *Abstract Syntax Tree*. Ini berbeda dengan *Parse Tree* yang menyimpan seluruh detail perbedaan *grammar*, AST merepresentasikan struktur logis program dengan lebih ringkas. Pada tahap ini, *semantic action* dijalankan pada *node parser tree* untuk membentuk *node* baru bagi AST. Hasil akhirnya adalah *Decorated AST*, dimana setiap *node* memiliki informasi minimal berupa tipe ekspresi dan referensi ke *symbol table* yang ada.

1.3. **Symbol Table**

Symbol Table adalah struktur data yang digunakan untuk menyimpan dan mengambil informasi mengenai deklarasi suatu simbol atau *identifier* selama proses kompilasi berlangsung. Informasi ini mencakup nama *identifier*, tipe data, dan lingkup (*scope*). Implementasi dari *symbol table* umumnya menggunakan struktur data *stack* untuk menangani *nesting scope*. Khusus untuk kompilator Pascal-S, terdapat tiga jenis tabel simbol yang dapat digunakan sebagai berikut.

1. *tab (Identifier Table)*, tabel yang digunakan untuk menyimpan informasi mengenai *identifier* umum seperti konstanta, variabel, prosedur, ataupun fungsi. Atribut penting dalam tabel ini meliputi:
 - a. id: nama *identifier*
 - b. link: *pointer* ke *identifier* sebelumnya dalam *scope* yang sama
 - c. obj: kelas objek seperti konstanta, variabel, dll.
 - d. type: tipe dasar *identifier* seperti integer, boolean, dll
 - e. ref: referensi ke tabel lain (*atab* atau *btab*) jika tipe komposit.
 - f. lev: tingkat lexical
 - g. adr: alamat atau *offset* memori
2. *btab (Block Table)*, tabel yang menyimpan informasi mengenai blok prosedur, fungsi, dan definisi tipe *record*. Atribut utamanya meliputi:
 - a. last: *pointer* ke *identifier* terakhir dalam blok tersebut.
 - b. lpar: *pointer* ke parameter terakhir (jika sebuah prosedur/fungsi)
 - c. psze: total ukuran parameter.
 - d. vsze: total ukuran variabel lokal.
3. *atab (Array Table)*, tabel yang menyimpan informasi spesifik mengenai *array*, seperti tipe indeks, batasan indeks, dan tipe elemen. Atributnya meliputi *xtyp* (tipe indeks), *etyp* (tipe elemen), *low* (batas bawah), *high* (batas atas), dan *elsz* (ukuran elemen).

1.4. **Visitor Pattern dan Type Checking**

Penelusuran AST dilakukan secara *top-down* menggunakan pola *Visitor* (*Visit Functions*). Setiap jenis *node* pada AST memiliki fungsi *visit_<node>* tersendiri yang bertugas untuk:

1. Memanggil fungsi *visit* pada *node* anak.
2. Mengakses atau memperbarui *symbol table*.
3. Menghitung atribut semantik.
4. Menambahkan anotasi pada *node*.

Proses validasi tipe dan lingkup (*Type and Scope Checking*) dilakukan dengan cara mengunjungi setiap *node* secara *depth-first*. Ketika memasuki blok kode baru, *scope* baru akan ditambahkan (*push*) ke *stack symbol table*, dan setiap deklarasi variabel akan dimasukkan ke dalam tabel tersebut. Setiap kali ditemukan penggunaan *identifier* dalam ekspresi, kompilator akan mencari (*lookup*) pada *symbol table* untuk memvalidasi keberadaan dan tipe datanya.

2. Perancangan dan Implementasi

Pada milestone ini, pengembangan difokuskan pada tahap *Semantic Analysis*, yang bertujuan untuk memastikan program yang ditulis memenuhi aturan semantik bahasa Pascal-S, serta mengubah representasi kode menjadi *Abstract Syntax Tree*.

2.1. Struktur Data *Abstract Syntax Tree* (AST)

Untuk merepresentasikan struktur dari kode program tanpa detail sintaks yang berlebihan (seperti tanda ‘;’ atau kurung), kami merancang kelas-kelas *ASTNode*. Seluruh *node* dalam AST mewarisi kelas dasar *ASTNode* yang menyediakan atribut dasar yang akan diisi pada tahap analisis semantik lebih lanjut seperti pada kode berikut.

```
class ASTNode:
    # Base class buat semua AST Nodes
    def __init__(self):
        self.type = None           # diset pas type checking
        self.scope_level = None
        self.tab_index = None     # reference ke symbol table

    def __repr__(self):
```

```
return f"{self.__class__.__name__}()"
```

Kami mendefinisikan berbagai kelas turunan untuk merepresentasikan konstruksi bahasa Pascal-S yang dapat dilihat pada tabel berikut:

Nama Kelas	Penjelasan
ProgramNode	Node untuk keseluruhan program
BlockNode	Node untuk block program (declarations + compound statement)
CompoundStatementNode	Node untuk compound statement
VarDeclNode	Node untuk deklarasi variabel
ConstDeclNode	Node untuk deklarasi konstanta
TypeDeclNode	Node untuk deklarasi tipe
ProcedureDeclNode	Node untuk deklarasi prosedur
FunctionDeclNode	Node untuk deklarasi fungsi
ParameterNode	Node untuk parameter prosedur/fungsi
TypeNode	Node untuk tipe dasar (integer, real, dll)
ArrayTypeNode	Node untuk tipe array
RangeNode	Node untuk range (low - high)
AssignNode	Node untuk assignment statement
IfNode	Node untuk if statement
WhileNode	Node untuk while statement
ForNode	Node untuk for statement
ProcedureFunctionCallNode	Node untuk prosedur/function call
BinOpNode	Node untuk binary operation
UnaryOpNode	Node untuk unary operation
NumberNode	Node untuk literal number

StringNode	Node untuk literal string
CharNode	Node untuk literal char
VarNode	Node untuk variabel
ArrayAccessNode	Node untuk akses elemen array
NoOpNode	Node untuk no operation (kosong)

Salah satu contoh implementasi node dapat dilihat pada potongan kode berikut.

```
class AssignNode(ASTNode):
    def __init__(self, target, value):
        super().__init__()
        self.target = target # VarNode
        self.value = value # Expression node

    def __repr__(self):
        return f"AssignNode(target = {self.target}, value = {self.value})"
```

2.2. Implementasi *Symbol Table*

Tabel simbol dirancang untuk menyimpan informasi mengenai *identifier* (variabel, konstanta, prosedur, dan fungsi) beserta atributnya. Sistem *symbol table* yang kami implementasikan menggunakan tiga struktur data utama untuk menangani *scope* dan tipe data yang kompleks:

1. TAB (*Identifier Table*), menyimpan entri simbol seperti nama, jenis objek, tipe data dan referensi
2. BTAB (*Block Table*), menyimpan informasi mengenai blok/scope saat ini untuk manajemen memori statis
3. ATAB (*Array Table*), menyimpan informasi spesifik untuk tipe data array seperti indeks dan tipe elemen

Untuk manajemen *scope*, kami menggunakan pendekatan *stack-based* dengan array bernama *display* untuk mengelola *lexical scope*, dengan beberapa method seperti *enter_block()* untuk menaikkan level *scope* dan menyiapkan pointer baru pada array, *exit_block()* untuk menurunkan level *scope* dan menghapus pointer dari array, dan *lookup()*

untuk mencari *identifier* dimulai dari *scope* terdalam hingga ke global. Kodenya dapat dilihat sebagai berikut.

```
class SymbolTables:
    def __init__(self):
        self.tab = []
        self.btab = [BlockTableEntry()] # block 0 = global block
        self.atab = []
        self.display = [0] # pointer to tab index
    start for each level
        self.level = 0 # lexical level

    def lookup(self, name):
        idx = self.display[self.level] # pointer ke start
        while idx != 0:
            entry = self.tab[idx - 1]
            if entry.identifier == name:
                return idx - 1
            idx = entry.link
        return None
```

2.3. Pembangunan AST

Pembangunan *Abstract Syntax Tree* dilakukan bersamaan dengan proses *parsing*. Dalam kode kami, kami memodifikasi *Recursive Descent Parser* yang telah dibuat sebelumnya. Sebelumnya, parser hanya bertugas untuk validasi urutan token, akan tetapi sekarang setiap fungsi dalam parser bertanggungjawab untuk mengembalikan *node* yang merepresentasikan struktur semantik dari kode tersebut.

Pada implementasi kami, kami menerapkan *Syntax-Directed Translation*, dimana setiap aturan produksi dalam *grammar* dikaitkan dengan aksi semantik yang membangun node AST. Setiap metode dalam kelas Parser sekarang memiliki *return value* berupa objek *ASTNode* atau list dari *ASTNode*, bukan lagi *None*. Sebagai contoh, pada *assignment statement*, Parser membaca *identifier* target dan ekspresi nilai, kemudian menggabungkannya menjadi objek *AssignNode* seperti kode berikut.

```
def assignment_statement(self):
    global sym, i
    i += 1
    self.tree_list.append("<assignment_statement>", i)

    target_token = self.accept_identifier()
    self.accept('ASSIGN_OPERATOR', ':=')
```



```

        value_expr = self.expression()

        i -= 1
        return
AssignNode(target=VarNode(name=target_token.value),
value=value_expr)

```

Salah satu tantangan utama dalam membangun AST untuk ekspresi matematis adalah menjaga *precedence* operasi, dimana kami menangani ini dengan struktur fungsi `expression` → `simple_expression` → `term` → `factor`. Fungsi `term` menangani operasi perkalian dan pembagian (prioritas paling tinggi), dan membangun pohon biner `BinOpNode` secara iteratif seperti kode berikut:

```

def term(self):
    global sym,i
    i += 1
    self.tree_list.append("<term>", i)
    left = self.factor()
    while ((sym.type == 'ARITHMETIC_OPERATOR' and sym.value
in ('*', '/')) or (sym.type == 'KEYWORD' and sym.value in
('bagi', 'mod', 'dan'))):
        if sym.type == 'ARITHMETIC_OPERATOR':
            op_token = self.accept('ARITHMETIC_OPERATOR',
sym.value)
        else:
            op_token = self.accept('KEYWORD', sym.value)

        right = self.factor()
        left = BinOpNode(left=left, op=op_token.value,
right=right)

    i -= 1
    return left

```

Untuk struktur kontrol seperti “jika” dan “selama”, parser menangkap kondisi dan blok kode yang dieksekusi, kemudian menyimpan dalam atribut *node* yang sesuai. Sebagai contoh, implementasi `if_statement` kami dapat dilihat sebagai berikut:

```

def if_statement(self):
    global sym,i
    i += 1
    self.tree_list.append("<if_statement>", i)

    self.accept('KEYWORD', 'jika')
    condition = self.expression()

```

```

        self.accept('KEYWORD', 'maka')

        then_stmt = None
        if sym.type == 'IDENTIFIER':
            next_token = self.peek_next_token()
            if next_token and next_token.type ==
'ASSIGN_OPERATOR':
                then_stmt = self.assignment_statement()
            else:
                then_stmt = self.procedure_function_call()
        else:
            if sym.value == 'mulai':
                then_stmt = self.compound_statement()
            elif sym.value == 'jika':
                then_stmt = self.if_statement()
            elif sym.value == 'selama':
                then_stmt = self.while_statement()
            elif sym.value == 'untuk':
                then_stmt = self.for_statement()
            else:
                pass

        else_stmt = None
        if sym.type == 'KEYWORD' and sym.value == 'selain_itu':
            self.accept('KEYWORD', 'selain_itu')
            if sym.type == 'IDENTIFIER':
                next_token = self.peek_next_token()
                if next_token and next_token.type ==
'ASSIGN_OPERATOR':
                    else_stmt = self.assignment_statement()
                else:
                    else_stmt = self.procedure_function_call()
            else:
                if sym.value == 'mulai':
                    else_stmt = self.compound_statement()
                elif sym.value == 'jika':
                    else_stmt = self.if_statement()
                elif sym.value == 'selama':
                    else_stmt = self.while_statement()
                elif sym.value == 'untuk':
                    else_stmt = self.for_statement()
                else:
                    pass

        i -= 1
        return IfNode(condition=condition,
then_statement=then_stmt, else_statement=else_stmt)

```

Dengan struktur yang kami implementasikan, hasil akhir dari pemanggilan fungsi parse() adalah sebuah ProgramNode yang akan menjadi akar dari *Abstract Syntax Tree* yang siap untuk diproses oleh Type Checker di tahap selanjutnya.

2.4. Type Checker

Type Checker bertanggung jawab untuk memvalidasi konsistensi tipe data dalam program Pascal-S. Proses ini dilakukan dengan melakukan traversal pada AST yang telah dibentuk, sambil memanfaatkan symbol table untuk menyimpan dan mengambil informasi tipe dari setiap identifier.

Type Checker diimplementasikan menggunakan Visitor Pattern, dimana setiap jenis node AST memiliki fungsi `visit_<NodeType>()` yang khusus menangani validasi tipe untuk node tersebut. Kelas `TypeChecker` memiliki atribut utama berupa objek `SymbolTables` untuk lookup dan validasi deklarasi identifier:

```
class TypeChecker:
    def __init__(self, symbol_tables: SymbolTables):
        self.symbol_tables = symbol_tables
        self.current_function = None

    def visit(self, node: ASTNode):
        if node is None:
            return None

        method_name = f'visit_{node.__class__.__name__}'
        visitor = getattr(self, method_name, self.generic_visit)

        result_type = visitor(node)
        if result_type is not None:
            node.type = result_type

        return result_type
```

Type checker terintegrasi erat dengan symbol table untuk menangani scope dan mencegah redeclaration. Saat memproses deklarasi variabel, type checker melakukan validasi tipe dan memasukkan identifier ke symbol table dengan linking yang tepat:

```
def visit_VarDeclNode(self, node: VarDeclNode):
    var_type = self.resolve_type(node.type_name)

    # Check if array type
    atab_ref = 0
```

```

actual_type = var_type
if isinstance(var_type, tuple):
    actual_type, atab_ref = var_type

for var_name in node.names:
    existing = self.symbol_tables.lookup(var_name)
    if existing is not None:
        entry = self.symbol_tables.tab[existing]
        if entry.lev == self.symbol_tables.level:
            self.error(f"Variable '{var_name}' already
declared in this scope", node)
            continue

        # Add to symbol table dengan linking ke block chain
        idx = self.symbol_tables.add_symbol(
            identifier=var_name,
            obj=ObjKind.VARIABLE,
            typ=actual_type,
            ref=atab_ref,
            nrm=1 # normal variable
        )

        # Update vsze (variable size) di btab untuk block saat
ini
        current_block_idx =
self.symbol_tables.display[self.symbol_tables.level]
        if current_block_idx < len(self.symbol_tables.btab):
            self.symbol_tables.btab[current_block_idx].vsze += 1

        node.type = actual_type
        node.scope_level = self.symbol_tables.level

    return actual_type

```

Untuk ekspresi biner, type checker memvalidasi kompatibilitas operand dengan operator dan melakukan type promotion bila diperlukan. Implementasi utama ada pada fungsi `get_binop_result_type()` yang menangani berbagai jenis operator:

```

def visit_BinOpNode(self, node: BinOpNode):
    left_type = self.visit(node.left)
    right_type = self.visit(node.right)

    result_type = self.get_binop_result_type(node.op, left_type,
right_type)

    node.type = result_type

```

```

        return result_type

def get_binop_result_type(self, op, left_type, right_type):
    if op in ('+', '-', '*', '/'):
        if left_type == TypeKind.UNKNOWN or right_type ==
TypeKind.UNKNOWN:
            return TypeKind.UNKNOWN

        if left_type not in (TypeKind.INTEGER, TypeKind.REAL):
            dummy_node =
type('', (object,), {'line':0, 'column':0})()
            self.error(f"Arithmetic operator '{op}' requires
numeric operands, got {TypeKind.to_string(left_type)}",
dummy_node)
            return TypeKind.UNKNOWN

        if right_type not in (TypeKind.INTEGER, TypeKind.REAL):
            dummy_node =
type('', (object,), {'line':0, 'column':0})()
            self.error(f"Arithmetic operator '{op}' requires
numeric operands, got {TypeKind.to_string(right_type)}",
dummy_node)
            return TypeKind.UNKNOWN

        # Result type promotion
        if left_type == TypeKind.REAL or right_type ==
TypeKind.REAL:
            return TypeKind.REAL
        else:
            return TypeKind.INTEGER

    elif op in ('bagi', 'mod'):
        if left_type != TypeKind.INTEGER or right_type !=
TypeKind.INTEGER:
            dummy_node =
type('', (object,), {'line':0, 'column':0})()
            self.error(f"Operator '{op}' requires integer
operands", dummy_node)
            return TypeKind.UNKNOWN
        return TypeKind.INTEGER

    elif op in ('dan', 'atau'):
        if left_type != TypeKind.BOOLEAN:
            dummy_node =
type('', (object,), {'line':0, 'column':0})()
            self.error(f"Logical operator '{op}' requires
boolean operands, got {TypeKind.to_string(left_type)}",
dummy_node)
            if right_type != TypeKind.BOOLEAN:
                dummy_node =
type('', (object,), {'line':0, 'column':0})()
                self.error(f"Logical operator '{op}' requires
boolean operands, got {TypeKind.to_string(right_type)}",

```

```

dummy_node)
    return TypeKind.BOOLEAN

    elif op in ('=', '<>', '<', '>', '<=', '>='):
        if left_type == TypeKind.UNKNOWN or right_type ==
TypeKind.UNKNOWN:
            return TypeKind.BOOLEAN

            if not self.check_type_compatibility(left_type,
right_type):
                dummy_node =
type('', (object,), {'line':0, 'column':0})()
                self.error(f"Cannot compare {left_type} with
{right_type}", dummy_node)

                return TypeKind.BOOLEAN

```

Plai yang di-assign kompatibel dengan tipe variabel target:

```

def visit_AssignNode(self, node: AssignNode):
    target_type = self.visit(node.target)

    value_type = self.visit(node.value)

    if not self.check_type_compatibility(target_type,
value_type):
        self.error(
            f"Type mismatch in assignment: cannot assign
{TypeKind.to_string(value_type)} to
{TypeKind.to_string(target_type)}", node
        )

        node.type = TypeKind.UNKNOWN

    return None

```

Fungsi `resolve_type()` menangani konversi dari nama tipe (string atau `ArrayTypeNode`) menjadi kode tipe internal. Untuk array, fungsi ini juga menambahkan entri ke array table dan mengembalikan tuple berisi tipe dan index ke atab:

```

def resolve_type(self, type_name):
    if isinstance(type_name, str):
        # Built-in types
        type_map = {

```

```

        'integer': TypeKind.INTEGER,
        'real': TypeKind.REAL,
        'boolean': TypeKind.BOOLEAN,
        'char': TypeKind.CHAR,
        'string': TypeKind.STRING
    }

    if type_name.lower() in type_map:
        return type_map[type_name.lower()]

    idx = self.symbol_tables.lookup(type_name)
    if idx is not None:
        entry = self.symbol_tables.tab[idx]
        if entry.obj == ObjKind.TYPE:
            return entry.type

    # Create a dummy node for error reporting
    dummy_node = type('', (object,), {'line':0, 'column':0})()
    self.error(f"Unknown type '{type_name}'", dummy_node)
    return TypeKind.UNKNOWN

elif isinstance(type_name, ArrayTypeNode):
    element_type = self.resolve_type(type_name.element_type)

    if hasattr(type_name.index_range, 'low') and
hasattr(type_name.index_range, 'high'):
        low_node = type_name.index_range.low
        high_node = type_name.index_range.high

        # Extract integer values from NumberNode
        if isinstance(low_node, NumberNode):
            low = int(low_node.value)
        else:
            low = low_node

        if isinstance(high_node, NumberNode):
            high = int(high_node.value)
        else:
            high = high_node

        # Calculate array size
        element_size = 1 # Simplified

        # Add to atab and return the index
        atab_index = self.symbol_tables.add_array_type(
            xtyp=TypeKind.ARRAY,
            etyp=element_type,
            eref=0,
            low=low,
            high=high,
            elsz=element_size
        )

```

```

        return (TypeKind.ARRAY, atab_index)
    else:
        dummy_node =
type('', (object,), {'line':0, 'column':0})()
        self.error(f"Invalid array index range", dummy_node)
        return TypeKind.UNKNOWN

    else:
        return TypeKind.UNKNOWN

```

Setiap kali menemukan reference ke variabel, type checker melakukan lookup di symbol table dan mendekorasi node dengan informasi tipe dan referensi:

```

def visit_VarNode(self, node: VarNode):
    idx = self.symbol_tables.lookup(node.name)

    if idx is None:
        self.error(f"Undeclared identifier '{node.name}'", node)
        node.type = TypeKind.UNKNOWN
        node.tab_index = None
        return TypeKind.UNKNOWN

    entry = self.symbol_tables.tab[idx]

    node.type = entry.type
    node.tab_index = idx
    node.scope_level = entry.lev

    return entry.type

```

Dengan pendekatan ini, type checker tidak hanya memvalidasi konsistensi tipe, tetapi juga mendekorasi AST dengan informasi tipe dan referensi ke symbol table yang akan berguna untuk tahap code generation selanjutnya.

3. Pengujian

3.1. Tes Scoping

Input	<pre>program SemanticScope; variabel angka : integer; prosedur CekLokal; variabel angka : integer; mulai angka := 100; writeln('Angka Lokal: ', angka); selesai; mulai angka := 1; CekLokal(); writeln('Angka Global: ', angka); selesai.</pre>																																																																																								
Output	a. Symbol Table <table><tr><th colspan="9">SYMBOL TABLE (TAB)</th></tr><tr><th>Idx</th><th>ID</th><th>Obj</th><th>Type</th><th>Ref</th><th>Nrm</th><th>Lev</th><th>Adr</th><th>Link</th></tr><tr><td>32</td><td>read</td><td>procedure</td><td>unknown</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>33</td><td>SemanticScope</td><td>program</td><td>unknown</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td></tr><tr><td>34</td><td>angka</td><td>variable</td><td>integer</td><td>0</td><td>1</td><td>0</td><td>0</td><td>33</td></tr><tr><td>35</td><td>CekLokal</td><td>procedure</td><td>unknown</td><td>0</td><td>0</td><td>0</td><td>0</td><td>34</td></tr><tr><td>36</td><td>angka</td><td>variable</td><td>integer</td><td>0</td><td>1</td><td>2</td><td>0</td><td>0</td></tr></table> b. Block Table <table><tr><th>Idx</th><th>Last</th><th>Lpar</th><th>Psze</th><th>Vsze</th></tr><tr><td>0</td><td>35</td><td>0</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>2</td><td>36</td><td>1</td><td>0</td><td>1</td></tr><tr><td>3</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table> c. Array Table (kosong) d. Abstract Syntax Tree	SYMBOL TABLE (TAB)									Idx	ID	Obj	Type	Ref	Nrm	Lev	Adr	Link	32	read	procedure	unknown	0	0	0	0	0	33	SemanticScope	program	unknown	0	1	0	0	0	34	angka	variable	integer	0	1	0	0	33	35	CekLokal	procedure	unknown	0	0	0	0	34	36	angka	variable	integer	0	1	2	0	0	Idx	Last	Lpar	Psze	Vsze	0	35	0	0	1	1	0	0	0	0	2	36	1	0	1	3	0	0	0	0
SYMBOL TABLE (TAB)																																																																																									
Idx	ID	Obj	Type	Ref	Nrm	Lev	Adr	Link																																																																																	
32	read	procedure	unknown	0	0	0	0	0																																																																																	
33	SemanticScope	program	unknown	0	1	0	0	0																																																																																	
34	angka	variable	integer	0	1	0	0	33																																																																																	
35	CekLokal	procedure	unknown	0	0	0	0	34																																																																																	
36	angka	variable	integer	0	1	2	0	0																																																																																	
Idx	Last	Lpar	Psze	Vsze																																																																																					
0	35	0	0	1																																																																																					
1	0	0	0	0																																																																																					
2	36	1	0	1																																																																																					
3	0	0	0	0																																																																																					

	<pre> Program('SemanticScope') → type:programnode(name='semanticscope') ├─ VarDecl('angka') → type:integer ├─ ProcedureDecl('CekLokal') → tab_index:35, lev:0, lev:0 ├─ Block │ └─ VarDecl('angka') → type:integer │ └─ compound_statement Block │ └─ Assign → type:unknown │ └─ target 'angka' → tab_index:36, lev:2, type:integer │ └─ Call('writeln') → tab_index:29, lev:0, type:unknown │ └─ String(Angka Lokal:) → type:string │ └─ Var('angka') → tab_index:36, lev:2, type:integer └─ Block └─ Assign → type:unknown └─ target 'angka' → tab_index:34, lev:0, type:integer └─ Call('CekLokal') → tab_index:35, lev:0, type:unknown └─ Call('writeln') → tab_index:29, lev:0, type:unknown └─ String(Angka Global:) → type:string └─ Var('angka') → tab_index:34, lev:0, type:integer </pre>
Keterangan	Compiler berhasil melakukan kompilasi. Output yang paling penting untuk diperhatikan ada di SYMBOL TABLE (TAB). Ada dua entri untuk identifier bernama angka. Satu entri memiliki Lev (Level) 0 (variabel global) dan entri lainnya memiliki Lev 1 (variabel lokal dalam prosedur CekLokal). Ini membuktikan bahwa Type Checker berhasil menangani konsep scope dan variable shadowing (variabel lokal menutupi variabel global dengan nama yang sama).

3.2. Tes Array

Input	<pre> program TesTabelLarik; variabel nilai_ujian : larik[1..10] dari integer; saldo : larik[0..5] dari integer; i : integer; mulai nilai_ujian[1] := 100; nilai_ujian[10] := 90; saldo[0] := 5000; saldo[5] := 1000; writeln('Nilai pertama: ', nilai_ujian[1]); writeln('Saldo akhir: ', saldo[5]); selesai. </pre>
Output	a. Symbol Table

SYMBOL TABLE (TAB)

Idx	ID	Obj	Type	Ref	Nrm	Lev	Adr	Link
28	PACKED	reserved	unknown	0	0	0	0	0
29	writeln	procedure	unknown	0	0	0	0	0
30	write	procedure	unknown	0	0	0	0	0
31	readln	procedure	unknown	0	0	0	0	0
32	read	procedure	unknown	0	0	0	0	0
33	TestTabellarik	program	unknown	0	1	0	0	0
34	nilai_ujian	variable	array	0	1	0	0	33
35	saldo	variable	array	1	1	0	0	34
36	i	variable	integer	0	1	0	0	35

b. Block Table

Idx	Last	Lpar	Psize	Vsize
0	36	0	0	3
1	0	0	0	0

c. Array Table

Idx	Xtyp	Etyp	Eref	Low	High	Elsz	Size
0	array	integer	0	1	10	1	10
1	array	integer	0	0	5	1	6

d. Abstract Syntax Tree

	<pre> Program('TesTabelLarik') → type:programnode(name='testabellarik') ├─ VarDecl('nilai_ujian') → type:array │ └─ type_name ArrayType │ └─ index_range Range │ ├── low 1 → type:integer │ └─ high 10 → type:integer ├─ VarDecl('saldo') → type:array │ └─ type_name ArrayType │ └─ index_range Range │ ├── low 0 → type:integer │ └─ high 5 → type:integer ├─ VarDecl('i') → type:integer └─ Block ├── Assign → type:unknown │ ├── target ArrayAccess → type:integer │ │ ├── array_var 'nilai_ujian' │ │ └─ index_expression 1 → type:integer │ └─ Assign → type:unknown │ ├── target ArrayAccess → type:integer │ │ ├── array_var 'nilai_ujian' │ │ └─ index_expression 10 → type:integer │ └─ Assign → type:unknown │ ├── target ArrayAccess → type:integer │ │ ├── array_var 'saldo' │ │ └─ index_expression 0 → type:integer │ └─ Assign → type:unknown │ ├── target ArrayAccess → type:integer │ │ ├── array_var 'saldo' │ │ └─ index_expression 5 → type:integer │ └─ Call('writeln') → tab_index:29, lev:0, type:unknown │ ├── String(Nilai pertama:) → type:string │ └─ ArrayAccess → type:integer │ ├── array_var 'nilai_ujian' │ └─ index_expression 1 → type:integer └─ Call('writeln') → tab_index:29, lev:0, type:unknown ├── String(Saldo akhir:) → type:string └─ ArrayAccess → type:integer ├── array_var 'saldo' └─ index_expression 5 → type:integer </pre>
Keterangan	Pada pengujian ini, proses kompilasi berjalan sukses dan menghasilkan output yang menonjol pada bagian ARRAY TABLE (ATAB). Karena kode ini mendefinisikan dua variabel array (nilai_ujian dan saldo), tabel array akan memuat dua entri terpisah yang mencatat spesifikasi memori mereka: baris pertama untuk nilai_ujian dengan rentang indeks 1 sampai 10 (Size 10), dan baris kedua untuk saldo dengan rentang 0 sampai 5 (Size 6). Selain itu, Decorated AST juga terbentuk, di mana setiap node akses array (seperti nilai_ujian[1]) telah diverifikasi oleh Semantic Analyzer untuk memastikan tipe data indeksnya valid (integer) dan operasi assignment-nya sesuai aturan tipe data.

3.3. Tes Error Duplikasi

Input	<pre>program ErrorDuplicate; variabel x : integer; x : real; mulai x := 10; selesai.</pre>
Output	<pre> COMPILATION FAILED Error Message: Semantic error: Type Error at line 5, column 5: Variable 'x' already declared in this scope Context: ----- 4 x : integer; 5 x : real; ^ HERE 6 -----</pre>
Keterangan	Proses berhenti di fase 3 dengan pesan error semantik. Output terminal menampilkan pesan "Undeclared variable". Hal ini terjadi karena variabel x dideklarasikan dua kali (sebagai integer dan kemudian sebagai real) dalam scope yang sama (blok variabel global), yang melanggar aturan keunikan nama variabel.

3.4. Tes Error Scoping

Input	<pre> program ErrorScope; variabel global : integer; prosedur CekLokal; variabel lokal : integer; mulai lokal := 99; selesai; mulai global := 1; lokal := 5; selesai. </pre>
-------	---

4. Kesimpulan dan Saran

4.1. Kesimpulan

Pengembangan fase Semantic Analysis pada kompilator Pascal-S telah berhasil dilaksanakan dengan pemenuhan seluruh kriteria teknis sebagai berikut:

1. Pembentukan Abstract Syntax Tree (AST): Parser berhasil dimodifikasi menggunakan metode Syntax-Directed Translation untuk membentuk AST yang merepresentasikan struktur logis program secara efisien.
2. Implementasi Symbol Table Lengkap: Struktur data manajemen memori telah memenuhi standar Pascal-S yang mencakup tiga tabel wajib: tab (identifier), btab (blok/scope), dan atab (array).
3. Mekanisme Type & Scope Checking: Validasi semantik berhasil diterapkan menggunakan Visitor Pattern untuk memeriksa kesesuaian tipe data dan aturan scoping secara depth-first.
4. Keluaran Decorated AST: Kompilator sukses menghasilkan Decorated AST yang telah dianotasi dengan informasi tipe data, referensi simbol, dan level scope.
5. Validitas Pengujian: Sistem terbukti mampu menangani berbagai kasus uji semantik dan melakukan penanganan kesalahan (error handling) yang informatif tanpa menyebabkan crash.

4.2. Saran

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, terdapat beberapa aspek yang dapat ditingkatkan untuk pengembangan compiler Pascal-S di masa mendatang:

1. Saat ini, parser akan berhenti ketika menemukan error pertama. Penambahan mekanisme error recovery akan memungkinkan parser untuk melanjutkan parsing dan melaporkan multiple errors sekaligus, sehingga programmer dapat memperbaiki beberapa kesalahan dalam satu iterasi kompilasi.
2. Type checker saat ini belum memvalidasi apakah array access berada dalam batas yang valid. Implementasi pengecekan batas array pada compile-time (untuk constant indices) dan runtime

(untuk variable indices) akan mencegah error akses memori yang tidak valid.

3. Pesan error saat ini masih cukup generik. Penambahan informasi seperti line number, column number, dan snippet kode yang bermasalah akan membantu programmer mengidentifikasi dan memperbaiki error dengan lebih cepat.
4. Implementasi saat ini menggunakan linear search pada symbol table. Penggunaan hash table atau dictionary akan meningkatkan performa lookup dari $O(n)$ menjadi $O(1)$, terutama untuk program dengan jumlah simbol yang besar.

5. Lampiran

Pembagian Tugas

NIM	Nama	Pembagian Tugas	Presentasi Kontribusi
13523047	Indah Novita Tangdililing	● Integrasi, Error Handling ● Laporan	20%
13523049	Muhammad Fithra Rizki	● Symbol Table ● Laporan	20%
13523053	Sakti Bimasena	● Type Checker ● Laporan	20%
13523055	Muhammad Timur Kanigara	● Base & AST Nodes ● Laporan	20%
13523113	Kefas Kurnia Jonathan	● AST Builder (ke parser) ● Laporan	20%

6. Referensi

Rila Mandala, Parsing dan Analisis Semantik. (Diakses via edunex).

Hopcroft et al, Introduction to Automata Theory, Languages, and Computation.